



Cahier des charges

Plateforme de pilotage de la qualité logicielle

Qualité produit • Qualité projet • Tests • Couverture • Go/No-Go • Assurance qualité

Élément	Valeur
Organisation	DHI
Version	1.0
Statut	Document de cadrage fonctionnel et technique
Objet	Conception et réalisation d'une plateforme centralisée de pilotage de la qualité des produits et projets logiciels

Finalité : disposer d'un système de référence permettant à DHI de démontrer, preuves et indicateurs à l'appui, le niveau de qualité d'un produit ou projet et de justifier les décisions de mise en production.

Synthèse exécutive

La plateforme cible doit centraliser la gouvernance de la qualité logicielle de DHI sur plusieurs produits, projets, versions et releases. Elle doit relier les fonctionnalités, exigences, risques, tests, preuves, incidents, anomalies, contrôles qualité et décisions Go/No-Go dans une chaîne de traçabilité unique.

Le cœur fonctionnel repose sur un référentiel de qualité configurable, des campagnes de tests versionnées, une couverture multidimensionnelle, la détection des trous de tests, un graphe de dépendances, la gestion multi-testeurs, des points critiques bloquants, le calcul de scores explicables et un mécanisme de Go-Live Readiness.

La solution ne doit pas être un simple outil de recette. Elle doit constituer un Quality Management System logiciel capable de piloter l'amélioration continue, la dette qualité, la testabilité et l'apprentissage issu des incidents de production.

Priorité	Périmètre
MUST	Produits/projets, rôles, fonctionnalités, exigences, cas de tests, campagnes, exécutions, preuves, anomalies, couverture, trous de tests, dépendances, score qualité, dashboard, rapports, Go/No-Go, audit.
SHOULD	Incidents, moteur de règles, automatisation par API, contrôles qualité, testabilité, dette qualité, alerting avancé, CI/CD.
COULD	Suggestions intelligentes de tests, analyse prédictive du risque, recommandations de régression, comparaison automatique des versions et produits.

1. Objet du document

Le présent cahier des charges définit les besoins fonctionnels, non fonctionnels et les principales règles de gestion d'une plateforme destinée à permettre à DHI de piloter, mesurer, contrôler et améliorer la qualité de ses produits et projets logiciels.

- définition des objectifs et critères de qualité ;
- mesure de la santé d'un projet ou d'un produit ;
- identification des éléments critiques et points à surveiller ;
- gestion des fonctionnalités et exigences ;
- conception, organisation et exécution des tests ;
- gestion des campagnes de tests ;
- mesure de la couverture et détection des trous de tests ;
- gestion des dépendances entre tests ;
- enregistrement des preuves et comportements observés ;
- prise en compte des besoins fonctionnels et non fonctionnels ;
- suivi des anomalies et incidents ;
- capitalisation des campagnes précédentes ;
- contrôles qualité et testabilité ;
- tableaux de bord, alertes et reporting ;
- préparation et décision de Go Live / No-Go Live ;
- traçabilité complète des décisions qualité.

La plateforme constitue le système de référence de la qualité logicielle de DHI.

2. Contexte et problématique

DHI possède ou développe plusieurs produits logiciels. Chaque produit peut connaître plusieurs projets, évolutions, versions ou mises en production. Les processus qualité deviennent difficiles à maîtriser lorsque les validations sont dispersées, que la couverture est incomplète ou que les critères de mise en production ne sont pas formalisés.

- les tests sont dispersés dans différents outils ou documents ;

- les équipes ne testent que certaines parties du produit ;
- les éléments critiques ne sont pas systématiquement contrôlés ;
- les résultats des campagnes précédentes sont difficilement réutilisables ;
- les incidents de production ne sont pas transformés en nouveaux tests ;
- les critères conduisant à un Go Live ou à un No-Go ne sont pas formalisés ;
- certaines fonctions critiques sont insuffisamment testées ;
- les tests ne tiennent pas compte de leurs dépendances ;
- les besoins non fonctionnels sont moins suivis que les fonctionnalités ;
- les preuves de conformité ou de non-conformité sont difficiles à retrouver ;
- plusieurs testeurs peuvent fournir des conclusions contradictoires ;
- la qualité globale d'un produit est difficile à mesurer objectivement.

La plateforme doit apporter une gouvernance homogène, mesurable et traçable de la qualité.

3. Objectifs généraux

1. Disposer d'une vue consolidée de la qualité de l'ensemble des produits et projets.
2. Définir ce que signifie « qualité » pour chaque produit.
3. Mesurer la qualité à partir d'éléments vérifiables.
4. Identifier immédiatement les risques et éléments critiques.
5. Garantir qu'une fonctionnalité importante ne puisse être oubliée lors des tests.
6. Augmenter progressivement la couverture des tests.
7. Industrialiser les campagnes de validation.
8. Capitaliser les tests existants.
9. Systématiser les tests de régression.
10. Intégrer les incidents et anomalies au processus d'amélioration.
11. Tester aussi bien les besoins fonctionnels que non fonctionnels.
12. Mesurer la testabilité, la documentation et la maintenabilité.
13. Fournir des éléments objectifs pour les décisions de Go Live.
14. Conserver l'historique des décisions et preuves.
15. Fournir des indicateurs exploitables par les responsables qualité, produit, projet et les décideurs.

4. Principes directeurs

4.1. La qualité ne se résume pas au nombre de tests réussis

Un produit ayant 100 % de succès sur un faible nombre de tests mais possédant de nombreuses fonctionnalités non testées ne doit pas être considéré comme ayant une excellente qualité.

- résultats des tests ;
- couverture ;
- criticité ;
- anomalies ;
- incidents ;
- points à surveiller ;
- critères non fonctionnels ;
- testabilité ;
- conformité aux contrôles qualité.

4.2. Score et autorisation de mise en production sont distincts

Exemple : score qualité 91/100 et couverture 95 %, mais un test de sécurité critique échoue → décision recommandée : NO-GO.

4.3. Tout résultat doit être traçable

- test ;
- scénario ;
- fonctionnalité ;
- testeur ;
- date ;
- environnement ;
- version ;
- preuves ;
- observations ;
- anomalies détectées.

4.4. Les incidents alimentent l'amélioration continue

- nouvelle exigence ;
- nouveau point à surveiller ;
- nouveau cas de test ;
- nouveau scénario ;
- nouvelle règle de contrôle ;
- adaptation d'un test existant.

5. Périmètre fonctionnel

Domaine	Fonction principale
Portefeuille	Produits, projets, versions et releases
Gouvernance qualité	Critères, objectifs et responsables
Features	Registre des fonctionnalités
Criticité	Éléments critiques et points à surveiller
Tests	Cas, scénarios, exécutions et preuves
Campagnes	Organisation des phases de validation
Couverture	Mesure de ce qui est testé ou non
Dépendances	Graphe et ordre d'exécution des tests
Non fonctionnel	Performance, charge, résilience, sécurité, etc.
Incidents	Capitalisation des incidents
Contrôle qualité	Audits et contrôles périodiques
Go Live	Décision Go / No-Go
Reporting	Rapports et tableaux de bord
Alerting	Alertes qualité
Historisation	Traçabilité des évolutions et décisions

6. Concepts métiers principaux

6.1. Produit

Un produit représente une solution logicielle exploitée ou commercialisée par DHI.

- identité
- description
- propriétaire
- responsable qualité
- versions
- fonctionnalités
- critères qualité
- points critiques
- projets associés
- historique qualité.

6.2. Projet

Un projet correspond à une initiative de création ou d'évolution d'un produit. Un produit peut être associé à plusieurs projets.

- équipe
- objectifs
- fonctionnalités
- exigences
- versions
- campagnes
- tests
- incidents
- points à surveiller
- score qualité.

6.3. Release / Version

Toute campagne et toute décision Go/No-Go doit pouvoir être associée précisément à une version ou release.

La qualité de la version N doit être distinguée de celle de la version N+1.

6.4. Fonctionnalité

La plateforme doit maintenir un registre officiel des fonctionnalités.

- identifiant
- titre
- description
- domaine
- propriétaire
- criticité
- statut
- exigences
- dépendances
- risques
- tests associés
- taux de couverture
- incidents associés.

6.5. Exigence

Une exigence représente un comportement, une propriété ou une contrainte attendue.

- fonctionnelle
- sécurité
- performance
- disponibilité
- ergonomie
- accessibilité
- maintenabilité
- compatibilité
- résilience
- observabilité
- documentation
- testabilité
- catégorie personnalisée.

6.6. Point à surveiller

Un point à surveiller représente une caractéristique dont la défaillance présente un risque significatif.

- description
- criticité
- justification
- responsable
- critères de validation
- tests
- état
- preuves
- recommandations
- impact Go/No-Go.

6.7. Campagne de tests

Une campagne représente un ensemble structuré de validations réalisées pour un objectif donné.

- release
- recette
- régression
- validation avant production
- audit
- performance
- sécurité
- autre.

6.8. Scénario de test

Le scénario matérialise un chemin fonctionnel ou technique à vérifier.

Exemple : paiement d'une commande → confirmation → génération de facture → notification du client.

6.9. Cas de test

Un scénario contient un ou plusieurs cas de test.

Le cas de test constitue l'unité de validation.

6.10. Exécution de test

Une exécution correspond à l'application d'un cas de test dans un contexte précis.

- version
- environnement
- testeur
- date. Un cas peut donc posséder plusieurs exécutions.

7. Acteurs et profils

Profil	Responsabilités principales
Administrateur	Configuration générale et gestion des droits
Responsable qualité	Gouvernance qualité, campagnes et validation
Quality Manager	Pilotage global et indicateurs
Responsable produit	Qualité du produit
Chef de projet	Qualité du projet
Product Owner	Fonctionnalités et critères d'acceptation
Développeur	Tests techniques et corrections
Testeur / QA	Exécution et validation
Expert métier	Validation fonctionnelle
Expert technique	Validation non fonctionnelle
Responsable sécurité	Contrôles de sécurité
Approver Go Live	Décision de mise en production
Observateur / Auditeur	Consultation et contrôle

La solution devra permettre la création de profils personnalisés.

8. Gestion des responsabilités qualité

BF-001 — Responsable qualité

Chaque produit et chaque projet doit pouvoir posséder au minimum un responsable qualité.

BF-002 — Équipe qualité

La plateforme doit permettre d'associer plusieurs collaborateurs à un produit ou projet.

BF-003 — Responsabilités

Les responsabilités pourront être affectées au niveau produit, projet, campagne, fonctionnalité, test, point critique, incident ou contrôle qualité.

BF-004 — Séparation des responsabilités

Lorsque nécessaire, DHI devra pouvoir exiger qu'un test soit validé par une personne différente de son auteur ou du développeur ayant réalisé la fonctionnalité.

9. Gestion des critères de qualité

BF-010 — Référentiel de critères

La solution doit permettre de créer un référentiel de critères qualité.

- conformité fonctionnelle ;
- stabilité ;
- performance ;
- fiabilité ;
- sécurité ;
- résilience ;
- maintenabilité ;
- testabilité ;
- documentation ;
- observabilité ;
- ergonomie ;
- accessibilité.

BF-011 — Pondération

Chaque critère doit pouvoir recevoir une pondération configurable.

Critère	Pondération indicative
Fonctionnel	30 %
Sécurité	20 %
Performance	15 %
Fiabilité	15 %
Maintenabilité	10 %
Documentation	5 %
Testabilité	5 %

BF-012 — Critères spécifiques

Chaque produit pourra ajouter ses propres critères.

BF-013 — Critères bloquants

Un critère pourra être marqué bloquant indépendamment de son poids dans le score.

10. Score qualité et santé du projet

BF-020 — Score global

La plateforme doit calculer un score de qualité pour un produit, un projet, une version, une release, une campagne et une fonctionnalité. Le score sera exprimé sur 100.

BF-021 — Indicateur de santé

- Sain ;
- À surveiller ;
- À risque ;
- Critique.

Les seuils devront être configurables.

BF-022 — Composition du score

Le calcul devra pouvoir intégrer : $Q = f(R, C, K, I, NF, T, CQ)$.

Variable	Signification
R	Résultats des tests
C	Couverture
K	Couverture des éléments critiques
I	Incidents et anomalies
NF	Exigences non fonctionnelles
T	Testabilité
CQ	Contrôles qualité

Les coefficients devront être configurables.

BF-023 — Explicabilité

Tout score doit être explicable. L'utilisateur doit pouvoir identifier les facteurs positifs et négatifs, les tests en échec, les fonctionnalités insuffisamment couvertes et les éléments critiques en défaut.

11. Registre des fonctionnalités

BF-030 — Référentiel exhaustif

La plateforme doit fournir un référentiel exhaustif des fonctionnalités d'un produit ou projet.

BF-031 — Traçabilité

Chaque fonctionnalité doit pouvoir être liée à des exigences, risques, scénarios, cas de tests, incidents, points à surveiller et versions.

BF-032 — Criticité

Chaque fonctionnalité pourra être classifiée : Faible, Moyenne, Haute ou Critique.

BF-033 — Couverture

La plateforme affichera pour chaque fonctionnalité : nombre de tests, tests exécutés, réussis, échoués, bloqués, couverture et dernière date de validation.

12. Typologie des tests

Chaque cas de test devra posséder un type. Le référentiel devra supporter au minimum :

- fonctionnel ;
- intégration ;
- end-to-end ;
- régression ;
- unitaire ;
- API ;
- interface ;
- sécurité ;
- performance ;
- charge ;
- stress ;
- endurance ;
- résilience ;
- compatibilité ;
- accessibilité ;
- ergonomie ;
- disponibilité ;
- récupération ;
- installation ;
- migration ;
- documentation ;
- testabilité.

Les administrateurs devront pouvoir ajouter leurs propres types.

13. Registre des règles de tests

BF-040 — Règles par type de test

Chaque type de test pourra être associé à un ensemble de règles obligatoires ou recommandées.

Exemple — Test de performance

- charge nominale ;
- charge maximale ;
- nombre d'utilisateurs ;
- durée ;
- temps de réponse attendu ;
- percentile P95/P99 ;
- taux d'erreur maximal ;
- consommation CPU ;
- mémoire ;
- débit.

Exemple — Test de sécurité

- scénario de menace ;

- niveau de risque ;
- résultat attendu ;
- preuve ;
- environnement ;
- outil utilisé.

Exemple — Test fonctionnel

- préconditions ;
- données ;
- étapes ;
- résultat attendu ;
- critères d'acceptation ;
- preuves.

Les règles pourront être obligatoires, recommandées ou informatives.

14. Gestion des cas de tests

Chaque cas de test devra comporter au minimum les champs suivants :

Champ	Obligatoire
Identifiant	Oui
Nom	Oui
Description	Oui
Type	Oui
Fonctionnalité ou exigence associée	Oui
Criticité	Oui
Préconditions	Selon type
Données d'entrée	Selon type
Étapes	Oui
Résultat attendu	Oui
Méthode d'évaluation	Oui
Tolérance	Selon type
Responsable	Oui
État	Oui
Version	Oui
Pièces justificatives	À l'exécution

15. Tests mesurables et comportementaux

15.1. Test mesurable

Exemple : le temps de réponse doit rester inférieur ou égal à 800 ms.

- métrique ;
- unité ;
- seuil ;
- valeur attendue ;
- valeur observée ;
- tolérance ;
- opérateur : <, ≤, =, ≥, >.

Le verdict pourra être calculé automatiquement.

15.2. Test comportemental

Exemple : lorsque l'utilisateur saisit un mot de passe incorrect, le système refuse la connexion et affiche le message approprié.

- comportement attendu ;
- comportement observé ;
- conformité ;
- commentaire ;
- preuve.

15.3. Verdict standard

Code	Libellé
PASS	Conforme
FAIL	Non conforme
BLOCKED	Bloqué
NOT RUN	Non exécuté
NOT APPLICABLE	Non applicable
PASS WITH RESERVATION	Conforme avec réserve

16. Gestion des preuves

BF-050 — Pièces justificatives

Toute exécution doit pouvoir recevoir une ou plusieurs preuves.

- capture d'écran ;
- vidéo ;
- fichier ;
- rapport ;
- log ;
- trace ;
- export ;

- document ;
- URL ;
- commentaire ;
- valeur mesurée.

BF-051 — Métadonnées

- auteur ;
- date ;
- test concerné ;
- exécution ;
- version ;
- environnement.

BF-052 — Preuve d'anomalie

Une anomalie déclarée durant un test doit pouvoir pointer vers les preuves établissant la non-conformité.

17. Enregistrement des comportements

La plateforme doit permettre d'enregistrer le comportement réel du système durant une validation. Cela pourra inclure :

- journaux applicatifs ;
- logs ;
- traces ;
- métriques ;
- captures ;
- vidéos ;
- chronologie des actions ;
- événements techniques ;
- requêtes/réponses ;
- observations du testeur.

Le système devra rapprocher ces éléments de l'exécution ayant généré le comportement. Les données sensibles, secrets, mots de passe et données personnelles devront pouvoir être masqués ou exclus des enregistrements.

18. Campagnes de tests

BF-060 — Création

- nom ;
- objectif ;
- produit ;
- projet ;
- version ;
- environnement ;
- responsable ;
- dates ;
- périmètre ;
- participants ;
- scénarios ;
- tests ;
- critères de sortie.

BF-061 — Duplication

Une nouvelle campagne devra pouvoir être créée à partir d'une campagne précédente.

- reprendre tous les tests ;
- sélectionner certains tests ;
- modifier les tests ;
- ajouter de nouveaux tests ;
- supprimer les tests devenus inutiles.

BF-062 — Versionnement

La modification d'un test réutilisé devra permettre de conserver l'ancienne définition afin de maintenir l'historique des campagnes passées.

BF-063 — Avancement

- nombre total de tests ;
- tests exécutés ;
- réussis ;
- échoués ;
- bloqués ;
- non exécutés ;
- couverture ;
- progression ;
- incidents ouverts ;
- points critiques non validés.

19. Tests de régression et rejeu**BF-070 — Marquage régression**

Un test précédemment réussi pourra être marqué comme test de régression.

BF-071 — Sélection intelligente du rejeu

Lors d'une nouvelle campagne, le système devra proposer les tests à rejouer en fonction :

- des fonctionnalités modifiées ;
- de leur criticité ;
- des dépendances ;
- des incidents passés ;
- des tests précédemment échoués ;
- des zones impactées.

BF-072 — Historique

Release 2.3 : réussi → Release 2.4 : réussi → Release 2.5 : échoué → régression détectée.

20. Dépendances et graphe des tests**BF-080 — Dépendances**

- doit être exécuté avant ;
- doit être exécuté après ;
- nécessite la réussite de ;
- bloque ;

- dépend fonctionnellement de.

BF-081 — Visualisation

```

Développeur / Test technique
↓
QA / Test API
↓
QA / Test fonctionnel
↓
Métier / Validation métier
↓
Test de régression
↓
Validation Go Live

```

BF-082 — Ordonnancement

Le moteur devra empêcher ou signaler l'exécution incohérente d'un test dont les prérequis ne sont pas satisfaits.

BF-083 — Cycles

Les dépendances circulaires devront être détectées.

21. Multi-testeurs

BF-090 — Affectation multiple

Un même cas de test pourra être affecté à plusieurs testeurs. Chaque testeur fournira son propre verdict.

BF-091 — Consolidation

- unanimité ;
- majorité ;
- validation par responsable ;
- règle « un FAIL implique FAIL » ;
- pondération selon le rôle.

Pour les éléments critiques, une validation négative non levée empêche le statut final PASS.

BF-092 — Divergence

Lorsqu'un testeur déclare PASS et un autre FAIL, le test devra afficher un état DIVERGENCE À ARBITRER. Une décision explicite sera nécessaire.

22. Couverture des tests

La plateforme doit fournir une mesure de couverture multidimensionnelle :

- couverture des fonctionnalités ;
- couverture des exigences ;
- couverture des scénarios ;
- couverture des risques ;
- couverture des éléments critiques ;
- couverture par type de test ;
- couverture fonctionnelle ;
- couverture non fonctionnelle.

Exemple : 240 fonctionnalités, 220 associées à au moins un test → couverture fonctionnelle = 91,7 %.

La plateforme devra distinguer : élément possédant un test ; élément testé dans la campagne courante ; élément ayant obtenu un résultat PASS.

23. Détection des trous de tests

La détection des trous constitue une fonctionnalité majeure.

BF-100 — Détection automatique

- fonctionnalité sans test ;
- exigence sans test ;
- élément critique sans test ;
- risque sans test ;
- fonctionnalité critique avec couverture insuffisante ;
- test jamais exécuté ;
- test non rejoué depuis plusieurs releases ;
- fonctionnalité modifiée sans test de régression ;
- exigence non fonctionnelle non évaluée.

BF-101 — Matrice de couverture

Fonctionnalité	Fonctionnel	Sécurité	Performance	Régression
Authentification	✓	✓	×	✓
Paieement	✓	✓	✓	✓
Export PDF	✓	×	×	✓

Les cases manquantes pourront déclencher des alertes selon les règles du produit.

BF-102 — Règles minimales

Une fonctionnalité critique pourra exiger obligatoirement un test fonctionnel, un test de régression, un test sécurité et un test performance. Ces règles devront être configurables.

24. Tests non fonctionnels

Performance

- temps de réponse ;
- débit ;
- latence ;

Charge

- utilisateurs simultanés ;
- transactions par seconde ;
- comportement sous forte charge ;

Stress

- point de rupture ;
- comportement en surcharge ;

Endurance

- stabilité sur plusieurs heures/jours ;
- fuite mémoire ;

Disponibilité

- taux de disponibilité ;
- interruption ;

Résilience

- perte de connexion ;
- panne d'un composant ;
- redémarrage ;
- fonctionnement dégradé ;

Sécurité

- authentification ;
- autorisation ;
- confidentialité ;
- intégrité ;
- vulnérabilités ;

Compatibilité

- navigateur ;
- OS ;
- mobile ;
- versions API ;

Maintenabilité

- documentation ;
- architecture ;
- dette technique ;
- facilité d'évolution ;

Testabilité

- isolation ;
- observabilité ;
- données de test ;
- automatisation ;
- reproductibilité ;

25. Tests de charge

La solution devra permettre l'enregistrement détaillé d'une campagne de charge. Informations possibles :

- nombre d'utilisateurs ;
- durée ;
- montée en charge ;
- scénario ;
- débit ;
- temps moyen ;

- médiane ;
- P95 ;
- P99 ;
- nombre d'erreurs ;
- taux d'erreur ;
- CPU ;
- mémoire ;
- réseau ;
- ressources ;
- anomalies détectées.

Des fichiers issus d'outils externes devront pouvoir être joints.

26. Testabilité du produit

La plateforme devra permettre de calculer ou d'évaluer un indice de testabilité.

Dimension	Exemple
Observabilité	Logs et métriques accessibles
Contrôlabilité	Possibilité de générer les états requis
Isolation	Composants testables indépendamment
Automatisation	Tests automatisables
Données	Données de test disponibles
Environnements	Environnement reproductible
Documentation	Comportements documentés
Reproductibilité	Anomalies facilement reproductibles

Un produit difficile à tester devra subir une pénalité dans certains indicateurs qualité ou générer une recommandation.

27. Documentation et maintenabilité

La qualité devra intégrer la capacité du logiciel à évoluer. La plateforme pourra mesurer :

- documentation utilisateur ;
- documentation technique ;
- documentation API ;
- architecture ;
- procédures d'installation ;
- procédures de déploiement ;
- procédures de reprise ;
- documentation des tests ;
- documentation des dépendances.

Un contrôle qualité pourra attribuer un score à chacune de ces dimensions.

28. Gestion des incidents

BF-110 — Registre des incidents

La plateforme devra maintenir un registre des incidents associés au produit, projet, release ou fonctionnalité.

BF-111 — Données incident

- date ;
- description ;
- gravité ;
- impact ;
- fonctionnalité ;
- version ;
- cause ;
- résolution ;
- preuves ;
- actions correctives.

BF-112 — Capitalisation

À la clôture d'un incident : « Existe-t-il désormais un test permettant d'éviter la répétition de cet incident ? »

- créer un test ;
- modifier un test ;
- créer un point à surveiller ;
- créer une règle ;
- associer un test existant.

BF-113 — Dette qualité

Un incident important ne disposant d'aucun test de non-régression devra pouvoir apparaître comme dette qualité.

29. Anomalies

Une anomalie pourra être créée depuis une exécution de test et contiendra :

- titre ;
- description ;
- gravité ;
- priorité ;
- test ;
- feature ;
- release ;
- preuves ;
- auteur ;
- responsable ;
- statut ;
- résolution.

Nouveau → Confirmé → Affecté → En correction → À retester → Fermé

ou

Rejeté / Non reproductible / Accepté comme risque

30. Points critiques et points à surveiller

La plateforme doit disposer d'un registre spécifique des points dont la défaillance représente un risque significatif ou inacceptable.

Exemple générique : pour un système contrôlé par logiciel, lors de l'activation d'un frein, un composant lourd ne doit en aucun cas poursuivre son déplacement. Un tel point peut être critique et bloquant.

- description ;
- contexte ;
- niveau de criticité ;
- conséquence possible ;
- responsable ;
- tests associés ;
- critères de validation ;
- recommandations ;
- statut.

31. Go Live / No-Go Live

Un module spécifique devra assister la décision de mise en production.

BF-120 — Session de validation

Une session Go Live sera créée pour un produit, un projet et une release.

BF-121 — Checklist

Critère	Statut exemple
Campagne terminée	✓
Tests critiques réussis	✓
Couverture ≥ 90 %	✓
Points critiques validés	✗
Incident bloquant ouvert	Non
Performance validée	✓
Sécurité validée	✓

BF-122 — Décision

- GO ;
- GO sous réserve ;
- NO-GO ;
- AJOURNÉ.

BF-123 — Gate bloquant

- test critique en échec ;
- point critique non évalué ;
- anomalie bloquante ouverte ;

- couverture inférieure au seuil critique ;
- contrôle obligatoire non réalisé.

BF-124 — Dérogation

- décideur ;
- justification ;
- risque accepté ;
- date ;
- éventuelle date d'expiration.

BF-125 — Historique

- participants ;
- indicateurs au moment de la décision ;
- résultats ;
- recommandations ;
- réserves ;
- décision ;
- décideurs.

32. Contrôles qualité

La plateforme devra permettre de créer des contrôles qualité indépendamment des campagnes de tests. Un contrôle contiendra :

- produit ;
- projet ;
- date ;
- contrôleur ;
- référentiel utilisé ;
- observations ;
- non-conformités ;
- recommandations ;
- score ;
- plan d'action.

Les contrôles permettront de suivre l'évolution de la qualité dans le temps.

33. Recommandations

La plateforme devra pouvoir enregistrer des recommandations liées à un test, une campagne, un incident, un contrôle, une fonctionnalité, un produit, un projet ou une décision Go Live.

- description ;
- priorité ;
- responsable ;
- date cible ;
- état ;
- justification ;
- preuve de résolution.

34. Alertes

La plateforme doit disposer d'un moteur d'alertes. Exemples :

- couverture inférieure à 80 % ;
- test critique en FAIL ;
- fonctionnalité critique sans test ;
- campagne en retard ;
- test bloqué depuis X jours ;
- incident majeur ouvert ;
- point critique non vérifié ;
- score qualité sous un seuil ;
- Go Live proche avec tests incomplets ;
- régression détectée ;
- contrôle qualité expiré.

Les alertes pourront être informatives, avertissement, critiques ou bloquantes. Les seuils devront être configurables.

35. Tableau de bord produit

- score qualité global ;
- état de santé ;
- évolution du score ;
- couverture ;
- anomalies ;
- incidents ;
- tests échoués ;
- tests bloqués ;
- éléments critiques ;
- points à surveiller ;
- dernières campagnes ;
- contrôles qualité ;
- dette qualité ;
- état Go/No-Go.

36. Tableau de bord projet

- progression ;
- features développées ;
- features testées ;
- couverture ;
- campagnes ;
- score ;
- anomalies ;
- risques ;
- incidents ;
- responsabilités ;
- points critiques ;
- readiness pour Go Live.

37. Visualisations

Graphe des dépendances de tests

- prérequis ;
- parallélisation ;
- chemin critique ;

- blocages.

Graphe d'évolution de la qualité

- score par jour ;
- campagne ;
- release.

Graphique de couverture

- testé ;
- non testé ;
- réussi ;
- échoué ;
- bloqué.

Matrice de couverture

Features × Types de tests.

Répartition des anomalies

- criticité ;
- statut ;
- fonctionnalité ;
- release.

38. Rapport de campagne

Chaque campagne doit générer un rapport contenant au minimum :

1. Identification de la campagne.
2. Version testée.
3. Environnement.
4. Objectifs.
5. Participants.
6. Périmètre.
7. Nombre total de tests.
8. Taux d'exécution.
9. Taux de succès.
10. Taux d'échec.
11. Tests bloqués.
12. Couverture.
13. Couverture des éléments critiques.
14. Anomalies.
15. Incidents.
16. Résultats non fonctionnels.
17. Points à surveiller.
18. Preuves.
19. Recommandations.
20. Conclusion qualité.
21. Avis Go/No-Go.

Le rapport devra être exportable, notamment en PDF.

39. Rapport qualité produit

- score actuel ;
- évolution ;
- principaux risques ;
- qualité fonctionnelle ;
- qualité non fonctionnelle ;
- couverture ;
- régressions ;
- incidents ;
- testabilité ;
- dette qualité ;
- contrôles ;
- recommandations.

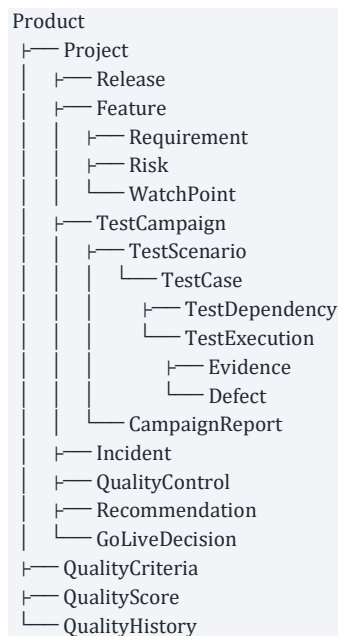
40. Historisation et audit

Toute modification significative doit être auditée. La plateforme doit permettre de savoir qui a modifié quoi, quand, avec l'ancienne valeur et la nouvelle valeur.

- cas de tests ;
- règles ;
- critères qualité ;
- seuils ;
- scores ;
- décisions Go/No-Go ;
- résultats de tests ;
- anomalies ;
- dérogations.

Les résultats historiques ne devront jamais être modifiés silencieusement.

41. Modèle logique principal



Entités transverses : User, Role, Team, Environment, TestType, TestRule, Alert, Notification, AuditLog, Attachment, Metric, Configuration.

42. Règles de gestion structurantes

ID	Règle
RG-001	Une fonctionnalité critique ne peut être considérée comme couverte sans test valide.
RG-002	Un test critique en échec doit influencer le statut global indépendamment du score moyen.
RG-003	Un test non exécuté ne doit jamais être considéré comme réussi.
RG-004	Une preuve doit être rattachée à une exécution déterminée.
RG-005	Toute campagne doit viser une version déterminée.
RG-006	La modification d'un test après son exécution ne doit pas modifier rétroactivement les résultats historiques.
RG-007	Les décisions Go/No-Go doivent rester historisées.
RG-008	Une dérogation doit être explicitement justifiée.
RG-009	Les éléments critiques doivent posséder un propriétaire.
RG-010	Un incident critique doit pouvoir être relié à au moins une action de prévention.
RG-011	Les règles de couverture doivent être configurables par produit.
RG-012	Le système doit détecter les cycles dans le graphe de dépendances.

43. Exigences non fonctionnelles de la plateforme

ID	Exigence	Description
ENF-001	Architecture	Architecture modulaire et évolutive ; approche API-first recommandée.
ENF-002	Web	Accessibilité via un navigateur Web moderne.
ENF-003	Responsive	Consultation exploitable sur ordinateur et tablette.
ENF-004	Performance	Cibles initiales : pages courantes < 2 s ; opérations simples < 1 s lorsque possible ; dashboards complexes < 5 s.
ENF-005	Volume	Plusieurs produits, centaines de projets, milliers de fonctionnalités, centaines de milliers de résultats de tests et montée en charge horizontale.
ENF-006	Disponibilité	Cible recommandée : au moins 99,5 % hors maintenances planifiées.
ENF-007	Sauvegarde	Mécanismes de sauvegarde et restauration.
ENF-008	Sécurité	HTTPS/TLS, chiffrement des données sensibles, contrôle d'accès, sessions sécurisées, journalisation, validation des entrées, protection contre les vulnérabilités Web.

ID	Exigence	Description
ENF-009	SSO	Intégration via OpenID Connect, OAuth 2.0 et SAML lorsque nécessaire.
ENF-010	RBAC	Droits par rôle et, si nécessaire, par produit/projet.
ENF-011	Audit	Traçabilité des opérations sensibles.
ENF-012	Intégrité	Résultats historiques non modifiables sans traçabilité.
ENF-013	Confidentialité	Accès limité aux projets autorisés.
ENF-014	Accessibilité	Utilisation au clavier et technologies d'assistance lorsque applicable ; ne pas communiquer un statut uniquement par la couleur.
ENF-015	Internationalisation	Architecture i18n ; français comme langue de référence minimale.
ENF-016	Observabilité	Logs, métriques, traces techniques et indicateurs de disponibilité.
ENF-017	Maintenabilité	Code modulaire, documenté, versionné, testé et soumis à analyse qualité.
ENF-018	Testabilité	La plateforme elle-même doit être conçue pour être facilement testable.
ENF-019	API	Principales fonctionnalités exposables par API pour intégration.
ENF-020	Exportabilité	Export des données importantes dans des formats standards.

44. Intégrations envisagées

L'architecture devra permettre à terme des intégrations avec :

- gestion de projet ;
- gestion des tickets ;
- CI/CD ;
- gestion de code source ;
- systèmes de supervision ;
- outils de tests automatisés ;
- scanners de sécurité ;
- outils de performance ;
- outils d'observabilité ;
- messagerie ;
- e-mail.

```

Pipeline CI/CD
↓
Tests automatisés
↓
API Qualité DHI
↓
TestExecution
↓
Recalcul du score
↓
Alerte éventuelle
↓
Go/No-Go

```

45. Automatisation des tests

La plateforme devra différencier test manuel, semi-automatique et automatisé. Pour un test automatisé, une API permettra à un système externe de transmettre :

- identifiant du test ;
- version ;
- date ;
- environnement ;
- statut ;
- valeurs mesurées ;
- logs ;
- preuves ;
- durée.

46. Moteur de règles

La solution devra intégrer un mécanisme permettant d'exprimer des règles sans développement spécifique pour les cas fréquents.

```
SI feature.criticité = "CRITIQUE"
ALORS couverture_minimale = 100 %

SI test.criticité = "CRITIQUE" ET test.resultat = "FAIL"
ALORS release.go_live = "BLOCKED"

SI incident.gravité = "CRITIQUE" ET incident.test_regression = NULL
ALORS générer_alerte("Dette qualité")
```

47. Mécanisme de readiness Go Live

Un indicateur spécifique de Go-Live Readiness devra être calculé à partir de la couverture, des tests critiques, incidents, anomalies, sécurité, performance, points à surveiller, contrôles qualité, validations et réserves.

GO-LIVE READINESS : 82 %

Tests critiques	100 %
Couverture	92 %
Sécurité	95 %
Performance	87 %
Incidents ouverts	70 %
Points à surveiller	75 %

Décision recommandée : NO-GO

Motif : 2 points critiques non validés.

48. Dette qualité

La solution devra matérialiser une dette qualité pouvant notamment provenir de :

- fonctionnalités non testées ;
- tests obsolètes ;
- incidents sans test de non-régression ;
- documentation absente ;
- faiblesse de testabilité ;
- anomalies acceptées ;
- recommandations non réalisées ;
- contrôles échus ;

- points à surveiller sans validation.

Un indicateur de dette qualité pourra être affiché par projet ou produit.

49. Recherche et filtres

Les utilisateurs devront pouvoir effectuer des recherches et filtres avancés, par exemple :

- tous les tests FAIL ;
- tous les tests critiques ;
- toutes les fonctionnalités sans test ;
- tous les incidents sans test de régression ;
- tous les tests bloqués ;
- toutes les campagnes d'une release ;
- tous les points critiques non validés ;
- toutes les recommandations en retard.

50. Notifications

Les utilisateurs devront pouvoir recevoir des notifications selon leur responsabilité. Événements possibles :

- affectation d'un test ;
- test échoué ;
- divergence de testeurs ;
- anomalie créée ;
- campagne démarrée ;
- campagne proche de son échéance ;
- point critique non validé ;
- décision Go/No-Go ;
- incident critique ;
- recommandation assignée.

51. Pages principales envisagées

1. Dashboard global qualité.
2. Portefeuille produits.
3. Fiche produit.
4. Fiche projet.
5. Registre des fonctionnalités.
6. Exigences.
7. Points à surveiller.
8. Référentiel de tests.
9. Scénarios.
10. Campagnes.
11. Écran d'exécution d'un test.
12. Matrice de couverture.
13. Graphe des dépendances.
14. Incidents.
15. Anomalies.
16. Contrôles qualité.
17. Rapports.
18. Go Live Center.

- 19. Alertes.
- 20. Référentiels et règles.
- 21. Administration.

52. Écran d'exécution d'un test

Cet écran constitue un élément central de la solution. Exemple de structure :

```

TC-1245 — Paiement par carte

Feature : Paiement
Criticité : CRITIQUE
Version : 4.12.0
Environnement : RECETTE

Préconditions
-----
...

Étapes
-----
1. ...
2. ...
3. ...

Résultat attendu
-----
...

Comportement observé
-----
...

Mesures
-----
Valeur attendue : < 800 ms
Valeur obtenue : 942 ms

Preuves : [capture.png] [logs.txt]
Verdict : [PASS] [FAIL] [BLOCKED]
Commentaire : ...

```

53. Dashboard exécutif

Le tableau de bord de pilotage global devra permettre à la direction ou au Quality Manager de visualiser rapidement :

Produits supervisés	12
Projets actifs	27
Score qualité moyen	86/100
Produits critiques	2
Campagnes actives	8
Tests en échec	142
Incidents critiques	3
Go Lives à venir	4

Chaque indicateur devra permettre un drill-down jusqu'au test ou à l'incident ayant produit l'indicateur.

54. Priorisation fonctionnelle proposée

MUST — Version initiale

- gestion produits/projets ;
- gestion utilisateurs et responsabilités ;
- registre des fonctionnalités ;
- exigences ;
- points critiques ;
- référentiel des tests ;
- scénarios et cas de tests ;
- campagnes ;
- exécution ;
- preuves ;
- anomalies ;
- dépendances ;
- couverture ;
- trous de tests ;
- score qualité ;
- tableaux de bord ;
- rapports ;
- Go/No-Go ;
- audit.

SHOULD

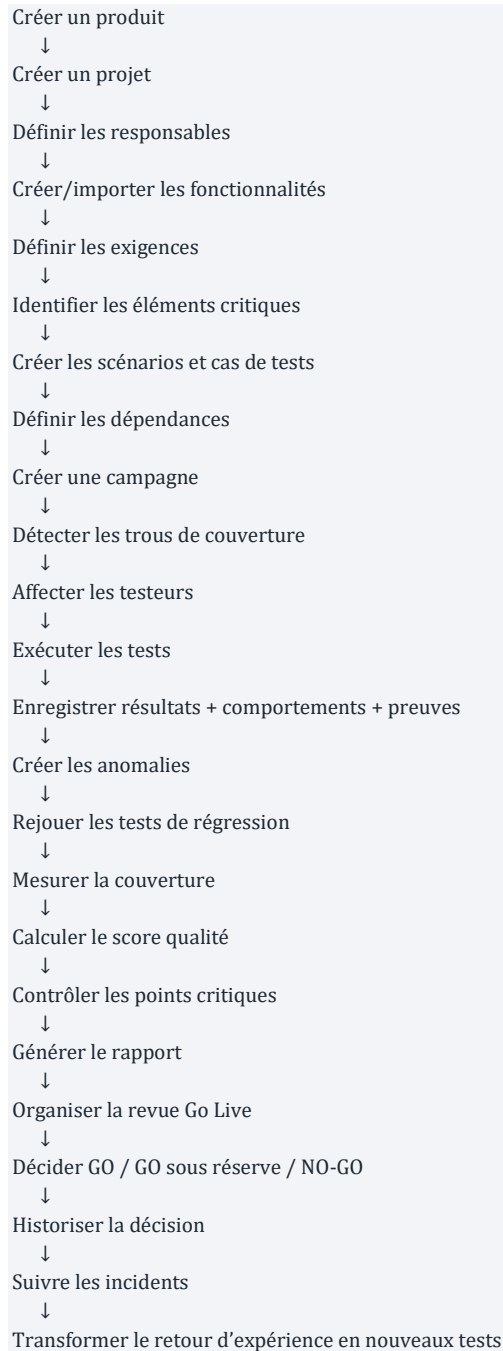
- incidents ;
- moteur de règles avancé ;
- automatisation par API ;
- contrôles qualité ;
- testabilité ;
- dette qualité ;
- alerting avancé ;
- intégrations CI/CD.

COULD

- recommandations automatiques de tests ;
- analyse intelligente des zones insuffisamment couvertes ;
- suggestions de tests de régression selon les changements ;
- analyse prédictive du risque d'une release ;
- comparaison automatique entre produits et versions.

55. Critères d'acceptation globaux

La solution sera considérée fonctionnellement conforme lorsque DHI pourra réaliser de bout en bout le processus suivant :



56. Indicateurs clés de performance qualité

KPI	Description
Quality Score	Niveau global de qualité
Test Coverage	Couverture des éléments testables
Critical Coverage	Couverture des éléments critiques
Pass Rate	Taux de tests réussis

KPI	Description
Execution Rate	Taux de tests exécutés
Failure Rate	Taux d'échec
Blocked Rate	Taux de tests bloqués
Regression Rate	Nombre de régressions
Defect Density	Anomalies rapportées au périmètre
Escaped Defects	Anomalies découvertes après mise en production
Incident Recurrence	Récurrence des incidents
Testability Index	Testabilité
Quality Debt	Dette qualité
Go-Live Readiness	Niveau de préparation à la mise en production

57. Vision cible

La plateforme ne doit pas devenir un simple outil où les équipes cliquent sur des boutons « vert » ou « rouge ». Elle doit progressivement constituer un véritable Quality Management System logiciel.

57.1. Avons-nous testé ce qui devait réellement être testé ?

Couverture, matrice fonctionnalités/tests, trous de tests, exigences et criticité.

57.2. Les éléments réellement importants fonctionnent-ils ?

Points critiques, tests critiques, règles bloquantes et anomalies.

57.3. Le produit fonctionne-t-il correctement au-delà de ses fonctionnalités ?

Performance, charge, sécurité, résilience, compatibilité, disponibilité, maintenabilité et testabilité.

57.4. Est-il suffisamment sûr de mettre cette version en production ?

Score qualité, Go-Live Readiness, critères bloquants, recommandations et revue Go/No-Go.

57.5. Apprenons-nous des défauts du passé ?

Incidents, régressions, historique, nouveaux tests, recommandations et dette qualité.

58. Résultat attendu

À terme, DHI doit disposer d'une plateforme permettant de passer d'une logique :

« *Nous avons effectué des tests.* »

à une logique :

« *Nous pouvons démontrer, preuves et indicateurs à l'appui, que le niveau de qualité attendu a été évalué, que les risques critiques sont maîtrisés et que la décision de mettre ou non le produit en production est justifiée et traçable.*

»

La plateforme devra donc servir simultanément de :

- référentiel qualité ;
- gestionnaire de tests ;
- système de couverture ;
- système de gouvernance ;
- registre des incidents ;
- outil d'aide à la décision ;
- outil de reporting ;
- système de capitalisation du retour d'expérience ;
- centre de décision Go Live / No-Go Live.

Elle constituera le socle central de gouvernance et d'assurance qualité logicielle de DHI.

Annexe A — Index des exigences identifiées

Cette annexe facilite la traçabilité des exigences pendant les ateliers de conception et les phases de recette.

ID	Intitulé	Type
BF-001	Responsable qualité	Fonctionnel
BF-002	Équipe qualité	Fonctionnel
BF-003	Responsabilités	Fonctionnel
BF-004	Séparation des responsabilités	Fonctionnel
BF-010	Référentiel de critères	Fonctionnel
BF-011	Pondération	Fonctionnel
BF-012	Critères spécifiques	Fonctionnel
BF-013	Critères bloquants	Fonctionnel
BF-020	Score global	Fonctionnel
BF-021	Indicateur de santé	Fonctionnel
BF-022	Composition du score	Fonctionnel
BF-023	Explicabilité	Fonctionnel
BF-030	Référentiel des fonctionnalités	Fonctionnel
BF-031	Traçabilité feature	Fonctionnel
BF-032	Criticité	Fonctionnel
BF-033	Couverture par fonctionnalité	Fonctionnel
BF-040	Règles par type de test	Fonctionnel
BF-050	Preuves	Fonctionnel
BF-051	Métadonnées des preuves	Fonctionnel
BF-052	Preuve d'anomalie	Fonctionnel
BF-060	Création de campagne	Fonctionnel
BF-061	Duplication	Fonctionnel
BF-062	Versionnement	Fonctionnel
BF-063	Avancement	Fonctionnel
BF-070	Test de régression	Fonctionnel
BF-071	Sélection du rejeu	Fonctionnel

ID	Intitulé	Type
BF-072	Historique de régression	Fonctionnel
BF-080	Dépendances	Fonctionnel
BF-081	Graphe	Fonctionnel
BF-082	Ordonnancement	Fonctionnel
BF-083	Cycles	Fonctionnel
BF-090	Multi-testeurs	Fonctionnel
BF-091	Consolidation	Fonctionnel
BF-092	Divergence	Fonctionnel
BF-100	Détection des trous	Fonctionnel
BF-101	Matrice de couverture	Fonctionnel
BF-102	Règles minimales	Fonctionnel
BF-110	Registre des incidents	Fonctionnel
BF-111	Données incident	Fonctionnel
BF-112	Capitalisation	Fonctionnel
BF-113	Dette qualité	Fonctionnel
BF-120	Session Go Live	Fonctionnel
BF-121	Checklist	Fonctionnel
BF-122	Décision	Fonctionnel
BF-123	Gate bloquant	Fonctionnel
BF-124	Dérogation	Fonctionnel
BF-125	Historique décision	Fonctionnel
ENF-001	Architecture	Non fonctionnel
ENF-002	Web	Non fonctionnel
ENF-003	Responsive	Non fonctionnel
ENF-004	Performance	Non fonctionnel
ENF-005	Volume	Non fonctionnel
ENF-006	Disponibilité	Non fonctionnel
ENF-007	Sauvegarde	Non fonctionnel

ID	Intitulé	Type
ENF-008	Sécurité	Non fonctionnel
ENF-009	SSO	Non fonctionnel
ENF-010	RBAC	Non fonctionnel
ENF-011	Audit	Non fonctionnel
ENF-012	Intégrité	Non fonctionnel
ENF-013	Confidentialité	Non fonctionnel
ENF-014	Accessibilité	Non fonctionnel
ENF-015	Internationalisation	Non fonctionnel
ENF-016	Observabilité	Non fonctionnel
ENF-017	Maintenabilité	Non fonctionnel
ENF-018	Testabilité	Non fonctionnel
ENF-019	API	Non fonctionnel
ENF-020	Exportabilité	Non fonctionnel
RG-001	Une fonctionnalité critique ne peut être considérée comme couverte sans test valide.	Règle de gestion
RG-002	Un test critique en échec doit influencer le statut global indépendamment du score moyen.	Règle de gestion
RG-003	Un test non exécuté ne doit jamais être considéré comme réussi.	Règle de gestion
RG-004	Une preuve doit être rattachée à une exécution déterminée.	Règle de gestion
RG-005	Toute campagne doit viser une version déterminée.	Règle de gestion
RG-006	La modification d'un test après son exécution ne doit pas modifier rétroactivement les résultats historiques.	Règle de gestion
RG-007	Les décisions Go/No-Go doivent rester historisées.	Règle de gestion
RG-008	Une dérogation doit être explicitement justifiée.	Règle de gestion
RG-009	Les éléments critiques doivent posséder un propriétaire.	Règle de gestion
RG-010	Un incident critique doit pouvoir être relié à au moins une action de prévention.	Règle de gestion
RG-011	Les règles de couverture doivent être configurables par produit.	Règle de gestion
RG-012	Le système doit détecter les cycles dans le graphe de dépendances.	Règle de gestion